

Azure Data Engineer

Complete Study Guide

All Skills • Concepts • Definitions • Examples

Target Certification: DP-203 | DP-700

1. Core Azure Services

Azure Data Factory (ADF)

A cloud-based ETL and data integration service that allows you to create data-driven workflows (pipelines) to orchestrate and automate data movement and transformation at scale.

Example:

Move customer transaction data nightly from an on-premises SQL Server to Azure Data Lake Storage using a Copy Activity inside an ADF pipeline triggered at midnight.

Code:

```
-- Key components:
Pipeline : Logical grouping of activities
Activity : A single step (Copy, Dataflow, Lookup)
Dataset : Named reference to data (source/sink)
Linked Svc : Connection string to data store
Trigger : Schedule / Tumbling Window / Event
```

Azure Synapse Analytics

An integrated analytics service combining enterprise data warehousing (dedicated SQL pools), big data analytics (serverless SQL / Apache Spark pools), and data integration — all in one workspace.

Example:

A retail company stores 10 years of sales data in a dedicated SQL pool and runs ad-hoc queries using serverless SQL pool on raw JSON files in ADLS without loading them first.

Code:

```
-- Pool types:
Dedicated SQL Pool : Provisioned DWH, MPP architecture
Serverless SQL Pool : Query files directly (pay per query)
Apache Spark Pool : Distributed processing (PySpark/Scala)
```

Azure Data Lake Storage Gen2 (ADLS Gen2)

A massively scalable object storage built on Azure Blob Storage with a hierarchical namespace (HNS) enabling file-system-like directory operations, fine-grained ACLs, and optimized analytics performance.

Example:

Raw log files land in a 'bronze' container, processed Parquet files move to 'silver', and aggregated data sits in 'gold' — a classic medallion architecture on ADLS Gen2.

Code:

```
-- Key features:
Hierarchical Namespace : Rename/move directories atomically
ACLs (POSIX-style) : File and folder-level permissions
Zones (containers) : bronze / silver / gold layers
Integration : Native with ADF, Databricks, Synapse
```

Azure Databricks

A fast, easy, and collaborative Apache Spark-based analytics platform optimized for Azure. Supports notebooks, ML workloads, Delta Lake, and integrates tightly with ADLS and Azure ML.

Example:

A data team writes PySpark notebooks in Databricks to clean raw IoT sensor data, apply transformations, and write Delta tables back to ADLS Gen2.

Code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("demo").getOrCreate()

df = spark.read.parquet("abfss://silver@storage.dfs.core.windows.net/sensors/")

df_clean = df.dropna().filter(df.temp > 0)

df_clean.write.format("delta").mode("overwrite").save("abfss://gold@.../sensors_clean/")
```

Azure SQL Database

A fully managed relational database-as-a-service (PaaS) based on SQL Server engine. Handles patching, backups, and high availability automatically.

Example:

An e-commerce app stores orders and customers in Azure SQL Database. ADF pulls this data hourly into Synapse for analytics.

Code:

```
-- Tiers:

General Purpose : Balanced compute/storage

Business Critical : High IOPS, in-memory OLTP

Hyperscale : Up to 100 TB, rapid scaling
```

Azure Event Hubs

A big data streaming platform and event ingestion service capable of receiving and processing millions of events per second. Acts as the 'front door' for real-time data pipelines.

Example:

A mobile app sends clickstream events to Event Hubs. Azure Stream Analytics reads from it, aggregates clicks per page per minute, and writes results to Cosmos DB.

Code:

```
-- Key concepts:

Namespace : Container for Event Hubs

Event Hub : Similar to a Kafka topic

Partition : Unit of parallelism

Consumer Group : Independent read position per app

Capture : Auto-save raw events to ADLS/Blob
```

Azure Stream Analytics

A real-time analytics service for processing fast-moving data streams from Event Hubs, IoT Hub, or Blob Storage using a SQL-like query language.

Example:

Calculate the average temperature from IoT sensors every 5 minutes and alert if it exceeds 80°C.

Code:

```
SELECT DeviceId, AVG(Temperature) AS AvgTemp, System.Timestamp AS WindowEnd
FROM IoTInput TIMESTAMP BY EventTime
GROUP BY DeviceId, TumblingWindow(minute, 5)
HAVING AVG(Temperature) > 80
```

2. SQL — Must-Know Concepts

Window Functions

Functions that perform calculations across a set of rows related to the current row without collapsing them into a single output row (unlike GROUP BY).

Example:

Rank each employee by salary within their department.

Code:

```
SELECT Name, Department, Salary,  
RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) AS Rank,  
LAG(Salary) OVER (PARTITION BY Department ORDER BY Salary DESC) AS PrevSalary  
FROM Employees
```

Common Table Expressions (CTEs)

A named temporary result set defined with the WITH clause that exists only for the duration of the query. Makes complex queries more readable and reusable within the same statement.

Example:

Find customers who placed more than 5 orders in the last 30 days.

Code:

```
WITH RecentOrders AS (  
  SELECT CustomerID, COUNT(*) AS OrderCount  
  FROM Orders  
  WHERE OrderDate >= DATEADD(day, -30, GETDATE())  
  GROUP BY CustomerID  
)  
SELECT c.Name, r.OrderCount  
FROM Customers c  
JOIN RecentOrders r ON c.ID = r.CustomerID  
WHERE r.OrderCount > 5
```

Query Optimization

Techniques to improve SQL query performance — reducing I/O, memory, and CPU usage.

Code:

```
-- Use indexes on filter/join columns  
-- Avoid SELECT *, fetch only needed columns  
-- Use EXISTS instead of IN for large subqueries  
-- Partition large tables and use partition pruning  
-- Use EXPLAIN / Query Execution Plan to find bottlenecks
```

Slowly Changing Dimensions (SCD)

A concept in data warehousing that handles how dimension data changes over time.

Example:

A customer moves cities. SCD Type 2 keeps the old city row (with end date) and adds a new row with the new city.

Code:

```
-- SCD Types:
Type 1 : Overwrite old value (no history)
Type 2 : Add new row with effective dates (full history)
Type 3 : Add new column for previous value (limited history)
```

3. Python for Data Engineering

Pandas — Data Manipulation

A Python library providing DataFrame and Series objects for fast, flexible data analysis and manipulation.

Example:

Load a CSV, filter rows, group by category and compute mean.

Code:

```
import pandas as pd
df = pd.read_csv("sales.csv")
df_filtered = df[df["Region"] == "North"]
result = df_filtered.groupby("Product")["Revenue"].mean().reset_index()
result.to_parquet("output.parquet", index=False)
```

Python Automation & Pipeline Scripts

Python scripts used to automate data engineering tasks such as triggering ADF pipelines, managing ADLS files, or calling REST APIs.

Example:

Trigger an ADF pipeline run programmatically.

Code:

```
from azure.mgmt.datafactory import DataFactoryManagementClient
from azure.identity import DefaultAzureCredential

credential = DefaultAzureCredential()
client = DataFactoryManagementClient(credential, subscription_id)
run = client.pipelines.create_run(rg, factory_name, pipeline_name)
print(f"Pipeline run ID: {run.run_id}")
```

4. PySpark & Apache Spark

Spark Architecture

Apache Spark uses a Driver + Executors model. The Driver plans the job and the Executors run tasks in parallel across a cluster. Data is processed in-memory using RDDs or DataFrames.

Code:

```
Driver Program
|-- SparkContext --> Cluster Manager (YARN/Kubernetes)
|-- Executor (Worker Node 1)
|-- Executor (Worker Node 2)
Each Executor has Tasks running on partitions of data.
```

Transformations vs Actions

Transformations are lazy operations that build a DAG (e.g. filter, select, join). Actions trigger actual computation and return results (e.g. count, show, write).

Code:

```
# Transformations (lazy – nothing runs yet)
df2 = df.filter(df.age > 25).select("name", "age")

# Action (triggers execution)
df2.show()
df2.count()
df2.write.parquet("output/")
```

Partitioning & Bucketing

Partitioning splits data into folders by column value for partition pruning. Bucketing distributes data into fixed buckets by hash for efficient joins.

Example:

Partition sales data by year and month so queries for a specific month scan only that folder.

Code:

```
df.write.partitionBy("year", "month").parquet("sales/")

# Bucketing (Spark SQL tables)
df.write.bucketBy(8, "customer_id").sortBy("customer_id") \
    .saveAsTable("sales_bucketed")
```


Spark Optimization Techniques

Key techniques to improve Spark job performance.

Code:

```
-- Broadcast joins: broadcast small table to avoid shuffle
from pyspark.sql.functions import broadcast
df_result = large_df.join(broadcast(small_df), "id")

-- Cache frequently reused DataFrames
df.cache()

-- Avoid wide transformations (groupBy, join) on unskewed keys
-- Use Kryo serialization for custom objects
-- Repartition before writing: df.repartition(10).write...
```

5. Delta Lake

What is Delta Lake?

An open-source storage layer built on Parquet that brings ACID transactions, schema enforcement, time travel, and scalable metadata handling to data lakes.

Example:

A pipeline writes to a Delta table. If a bad batch arrives, you can roll back to yesterday's version without re-running the whole pipeline.

Code:

```
-- Delta table = Parquet files + _delta_log/ (transaction log)
-- Supports: INSERT, UPDATE, DELETE, MERGE on data lake files
```

ACID Transactions

Atomicity, Consistency, Isolation, Durability — guarantees that data operations either fully succeed or fully fail, preventing partial writes and corruption.

Example:

Two pipelines writing simultaneously won't corrupt the table — Delta uses optimistic concurrency control.

Time Travel

Query older versions of a Delta table using a version number or timestamp.

Example:

Audit what the table looked like before yesterday's bad data load.

Code:

```
-- By version
df = spark.read.format("delta").option("versionAsOf", 3).load("path/")

-- By timestamp
df = spark.read.format("delta") \
    .option("timestampAsOf", "2024-01-15") \
    .load("path/")
```

MERGE (Upsert)

Combines INSERT and UPDATE in one atomic operation — insert new records, update existing ones.

Example:

Sync incoming CDC (Change Data Capture) records into a Delta table.

Code:

```
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "delta/customers/")
dt.alias("target").merge(
    updates_df.alias("src"), "target.id = src.id"
).whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()
```

Schema Evolution & Enforcement

Schema enforcement rejects writes with mismatched schemas. Schema evolution (`mergeSchema`) allows new columns to be added automatically.

Code:

```
# Enforce schema (default) – raises error if schema doesn't match
df.write.format("delta").mode("append").save("path/")

# Allow schema evolution
df.write.format("delta").option("mergeSchema", "true") \
    .mode("append").save("path/")
```

6. Data Modeling Concepts

Star Schema

A data warehouse design with one central fact table surrounded by denormalized dimension tables. Optimized for read performance and simple queries.

Example:

FactSales connects to DimDate, DimProduct, DimCustomer, DimStore — one join per dimension.

Code:

```
FactSales
SalesID (PK), DateID (FK), ProductID (FK),
CustomerID (FK), StoreID (FK), Quantity, Revenue

DimProduct
ProductID (PK), Name, Category, SubCategory, Brand
```

Snowflake Schema

An extension of star schema where dimension tables are normalized into multiple related tables. Saves storage but requires more joins.

Example:

DimProduct splits into DimProduct -> DimCategory -> DimSubCategory.

Medallion Architecture (Bronze/Silver/Gold)

A data lakehouse design pattern organizing data into three quality layers for progressive refinement.

Code:

```
Bronze : Raw ingested data – no transformations, append-only
(CSV, JSON, Parquet as-is from source)

Silver : Cleaned, validated, deduplicated data
(standardized schemas, null handling, joins)

Gold : Business-level aggregates, ready for reporting
(KPIs, daily summaries, star schema tables)
```

ETL vs ELT

ETL (Extract-Transform-Load) transforms data before loading into the target. ELT (Extract-Load-Transform) loads raw data first, then transforms inside the target system.

Example:

ELT is common in modern cloud DWHs — load raw data into Synapse or BigQuery, then transform with SQL/Spark.

7. Security & Governance

Role-Based Access Control (RBAC)

Azure's authorization system that grants access to Azure resources based on assigned roles rather than individual permissions.

Example:

Assign the 'Storage Blob Data Contributor' role to an ADF Managed Identity so it can write to ADLS without a password.

Code:

```
-- Common roles for data engineering:  
Storage Blob Data Reader : Read ADLS files  
Storage Blob Data Contributor : Read + Write ADLS  
Synapse Administrator : Full Synapse access  
Data Factory Contributor : Manage ADF pipelines
```

Managed Identity

An Azure AD identity automatically managed by Azure for a service (e.g. ADF, Databricks). Eliminates the need to manage credentials — services authenticate using their identity.

Example:

ADF uses its System-Assigned Managed Identity to connect to Key Vault without storing any passwords.

Azure Key Vault

A cloud service for securely storing and accessing secrets (passwords, connection strings, API keys), certificates, and cryptographic keys.

Example:

Store the SQL Server password in Key Vault, then reference it in ADF linked service as @Microsoft.KeyVault(SecretUri=...)

Microsoft Purview

A unified data governance service for data discovery, classification, lineage tracking, and cataloging across on-premises and cloud data sources.

Example:

Scan all ADLS containers to auto-classify columns containing PII (e.g. email, SSN), track data lineage from source to Power BI report.

Code:

```
-- Key features:  
Data Catalog : Search and discover data assets  
Data Lineage : Visual source-to-destination flow  
Classifications: Auto-tag sensitive data (PII, GDPR)  
Glossary : Business terms linked to data assets
```

8. DevOps & CI/CD for Data Pipelines

Git-based Deployment for ADF

Azure Data Factory integrates with Azure DevOps Git or GitHub repos. Changes are made in feature branches and published to production via CI/CD pipelines using ARM templates.

Example:

Developer creates a new ADF pipeline in a feature branch, raises a PR, and upon merge to main, a DevOps pipeline auto-deploys to production ADF.

Code:

```
-- ADF CI/CD flow:
1. Connect ADF studio to Azure DevOps repo
2. Develop in feature branch
3. Merge to collaboration branch (e.g. main)
4. Publish generates ARM templates in adf_publish branch
5. Release pipeline deploys ARM to Prod ADF
```

Infrastructure as Code (IaC)

Managing and provisioning Azure infrastructure through code (ARM templates, Bicep, Terraform) rather than manual portal clicks — ensures repeatability and version control.

Example:

Provision an entire Synapse workspace, ADLS account, and Databricks cluster from a single Terraform script.

Code:

```
# Terraform example
resource "azurerm_data_factory" "adf" {
  name = "my-adf"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  identity { type = "SystemAssigned" }
}
```

Pipeline Monitoring & Alerting

Track pipeline runs, set up alerts on failures, and monitor data quality using Azure Monitor, Log Analytics, and ADF/Synapse built-in monitoring.

Example:

Set an Azure Monitor alert to send an email if an ADF pipeline fails more than twice in one hour.

Code:

```
-- Tools:
ADF Monitor Tab : Pipeline run history, activity details
Azure Monitor : Metrics, alerts, dashboards
Log Analytics : Query diagnostic logs with KQL
Databricks Jobs UI : Cluster and job run history
```

9. Real-Time & Streaming Data

Batch vs Streaming Processing

Batch processing handles large volumes of data at scheduled intervals. Streaming processes data continuously as it arrives with low latency.

Example:

Batch: run a daily job to aggregate yesterday's sales. Stream: alert within seconds when a fraudulent transaction is detected.

Tumbling, Sliding & Session Windows

Window types used in stream processing to group events over time.

Code:

```
Tumbling : Fixed, non-overlapping windows
```

```
e.g. every 5 min: [0-5], [5-10], [10-15]
```

```
Sliding : Overlapping windows that slide by a step
```

```
e.g. 5-min window sliding every 1 min
```

```
Session : Dynamic windows based on activity gaps
```

```
e.g. group events until 10-min idle gap
```

Structured Streaming (Spark)

A scalable, fault-tolerant streaming API built on Spark SQL engine — treats a stream as an unbounded table.

Example:

Read live events from Event Hubs and write aggregated results to a Delta table every 30 seconds.

Code:

```
df_stream = spark.readStream.format("eventhubs") \
    .options(**ehConf).load()
```

```
agg = df_stream.groupBy(
    window("timestamp", "30 seconds"), "device_id"
).count()
```

```
agg.writeStream.format("delta") \
    .outputMode("update") \
    .option("checkpointLocation", "/chk/") \
    .start("abfss://gold@storage.../device_counts/")
```

10. Certifications & Learning Path

Certification	Level	Focus Areas
DP-900	Beginner	Core data concepts, Azure storage, analytics basics
DP-203	Associate	ADF, Synapse, Databricks, ADLS, streaming, security
DP-700	Associate	Microsoft Fabric, Lakehouse, Data Factory in Fabric
AZ-900	Beginner	Azure fundamentals — good prerequisite

Recommended Learning Order

- **Step 1:** Strong SQL fundamentals — window functions, CTEs, optimization
- **Step 2:** Python basics + Pandas for data manipulation
- **Step 3:** Core Azure services — ADF, ADLS Gen2, Azure SQL
- **Step 4:** Synapse Analytics — SQL pools, serverless queries
- **Step 5:** PySpark + Azure Databricks notebooks
- **Step 6:** Delta Lake — ACID, time travel, MERGE, schema evolution
- **Step 7:** Security & governance — RBAC, Key Vault, Managed Identity, Purview
- **Step 8:** CI/CD for data pipelines — Git, Azure DevOps, ARM templates
- **Step 9:** Streaming — Event Hubs, Stream Analytics, Structured Streaming
- **Step 10:** Prepare and pass DP-203, then DP-700

Good luck on your Azure Data Engineer journey!